

Software Design Patterns Interview Guide

Selection rules, Python examples, trade-offs, and low-level design

Reusable engineering knowledge - no candidate or company identifiers

Software Engineering Knowledge Hub

Contents

Software Design Patterns for Python and Backend Interviews	3
Recommended Study Flow	3
Foundations	3
Creational Patterns	3
Structural Patterns	5
Behavioral Patterns	6
Modern Backend Patterns	8
High-Value Comparisons	10
LLD Case Study: Extensible Transaction Processing	10
Common Red Flags	11
60-Second Answer Template	12
Quick Revision Checklist	12
Further Reading	12

Software Design Patterns for Python and Backend Interviews

Purpose: Provide a practical, interview-focused reference for the 23 Gang of Four patterns and modern backend patterns. **Use this file for:** pattern selection, Python examples, low-level design, comparisons, trade-offs, and quick revision.

Recommended Study Flow

1. Identify whether the changing force is **creation**, **structure**, or **interaction**.
2. Explain a pattern through intent, participants, one concrete example, and one trade-off.
3. Implement only the collaboration that matters; avoid ceremonial abstractions.
4. Compare neighboring patterns because interviews often test selection rather than recall.
5. Combine patterns in one small LLD, then explain why every pattern earns its complexity.

Foundations

A design pattern is a reusable arrangement of responsibilities for a recurring design problem. It is not finished code and not an algorithm.

Concept	Question answered	Examples
Principle	What quality should guide the decision?	SOLID, DRY, KISS, YAGNI
Design pattern	How can recurring responsibilities be arranged?	Strategy, Adapter, Observer
Architecture	How do major components and data flows fit?	Layered, event-driven, hexagonal
Algorithm	Which computational steps solve the input problem?	Binary search, merge sort

Selection Workflow

Ask:

1. What changes independently?
2. Where is the conditional or coupling pressure?
3. Is the change about object creation, object assembly, or object interaction?
4. Can a function, protocol, composition, or dependency injection solve it more simply?
5. What new indirection, state, or operational cost will the pattern add?

Strong interview line:

I use a pattern when it isolates a real source of change or protects an invariant. I do not introduce one merely to name it in the design.

Creational Patterns

1. Singleton

Intent: Ensure one instance and provide controlled access to it.

Backend use: Process-local configuration registry or metrics collector.

Trade-off: Hidden global state makes tests, concurrency, and multi-process deployment harder. In FastAPI, application lifespan plus dependency injection is usually clearer.

```
class Settings:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance
```

One instance per Python process is not one instance across containers, pods, or machines.

2. Factory Method

Intent: Define an object-creation interface while allowing a subclass or creator function to choose the concrete type.

```
from typing import Protocol

class Notifier(Protocol):
    def send(self, message: str) -> None: ...

def create_notifier(channel: str) -> Notifier:
    if channel == "email":
        return EmailNotifier()
    if channel == "sms":
        return SmsNotifier()
    raise ValueError(f"unsupported channel: {channel}")
```

Use when callers should depend on a stable interface rather than constructors and selection logic.

3. Abstract Factory

Intent: Create a compatible family of related objects without naming concrete classes.

Example: A cloud factory creates storage, queue, and secret-store adapters for AWS or Azure. The selected family remains internally compatible.

Trade-off: Adding a new product type to every family can be expensive.

4. Builder

Intent: Construct a complex object step by step while separating construction from representation.

```
class QueryBuilder:
    def __init__(self):
        self._filters: list[str] = []
        self._limit = 100

    def where(self, expression: str) -> "QueryBuilder":
        self._filters.append(expression)
        return self

    def limit(self, value: int) -> "QueryBuilder":
        self._limit = value
        return self

    def build(self) -> dict:
        return {"filters": tuple(self._filters), "limit": self._limit}
```

Prefer a dataclass/Pydantic model when construction is not genuinely complex.

5. Prototype

Intent: Create an object by cloning a configured prototype.

Example: Copy a baseline workflow configuration and customize a few fields.

Trade-off: Shallow copies share nested mutable objects; deep copies can be expensive or invalid for sessions, sockets, locks, and other resources.

Structural Patterns

6. Adapter

Intent: Convert one interface into another expected by the client.

```
class LegacyRiskApi:
    def score(self, cents: int) -> int:
        ...

class RiskClientAdapter:
    def __init__(self, legacy: LegacyRiskApi):
        self.legacy = legacy

    def risk_score(self, amount: float) -> float:
        return self.legacy.score(round(amount * 100)) / 100
```

Useful at API-version, vendor, or legacy-system boundaries.

7. Bridge

Intent: Separate an abstraction from its implementation so both dimensions can vary independently.

Example: Report types vary independently from delivery channels. Report delegates delivery to an injected `DeliveryChannel`.

Trade-off: Adds value only when both dimensions genuinely vary.

8. Composite

Intent: Treat individual objects and object groups uniformly through a tree structure.

Examples: File/folder trees, nested policy rules, UI components, organization hierarchies.

```
class Rule:
    def evaluate(self, context: dict) -> bool:
        raise NotImplementedError

class AllRules(Rule):
    def __init__(self, children: list[Rule]):
        self.children = children

    def evaluate(self, context: dict) -> bool:
        return all(child.evaluate(context) for child in self.children)
```

9. Decorator

Intent: Add behavior by wrapping an object without modifying the wrapped class.

```
class TracedPaymentGateway:
    def __init__(self, wrapped, tracer):
        self.wrapped = wrapped
        self.tracer = tracer

    async def authorize(self, payment):
        with self.tracer.start_as_current_span("authorize-payment"):
            return await self.wrapped.authorize(payment)
```

Use for tracing, caching, retry, authorization, and metrics when wrapper order and error semantics remain clear.

10. Facade

Intent: Expose a simplified interface over a complicated subsystem.

Example: CheckoutFacade.checkout() coordinates inventory, payment, order persistence, and notification behind one application-facing operation.

Trade-off: A facade can become a god object if it absorbs business rules instead of coordinating them.

11. Flyweight

Intent: Share immutable intrinsic state among many logical objects to reduce memory use.

Example: Reuse parsed rule definitions while keeping request-specific evaluation context separate.

Trade-off: Separating intrinsic and extrinsic state increases complexity and may not be worthwhile until profiling proves a memory problem.

12. Proxy

Intent: Control access to another object while preserving its interface.

Examples: Authorization proxy, remote proxy, lazy-loading proxy, caching proxy.

Unlike an Adapter, a Proxy preserves the conceptual interface; unlike a Decorator, its primary purpose is access/control rather than adding optional behavior.

Behavioral Patterns

13. Observer

Intent: Notify registered dependents when an in-process subject changes.

Example: A domain object notifies local observers that a state transition occurred.

Trade-off: Notification order, failure isolation, memory leaks, and re-entrant updates need explicit policy. For distributed systems, durable publish-subscribe is normally a different architectural mechanism.

14. Strategy

Intent: Encapsulate interchangeable algorithms behind one interface.

```
from typing import Protocol

class PricingStrategy(Protocol):
    def price(self, subtotal: int) -> int: ...

class StandardPricing:
    def price(self, subtotal: int) -> int:
        return subtotal

class PromotionalPricing:
    def price(self, subtotal: int) -> int:
        return max(0, subtotal - 500)
```

Use when a conditional selects an algorithm and new variants should not modify the caller.

15. Command

Intent: Represent a request as an object so it can be queued, retried, logged, authorized, or undone.

```
from dataclasses import dataclass
```

```
@dataclass(frozen=True)
class TransferCommand:
    operation_id: str
    source_account: str
    destination_account: str
    amount_cents: int
```

Distributed commands need idempotency, durable state, authorization, and observable outcomes.

16. Chain of Responsibility

Intent: Pass a request through an ordered chain of handlers until one handles or rejects it.

Examples: Middleware, validation pipeline, fraud rules, support escalation.

Trade-off: Handler order becomes behavior. Trace which handler made the decision.

17. Template Method

Intent: Define an algorithm skeleton in a base class while allowing selected steps to vary.

```
class ImportPipeline:
    def run(self, source):
        rows = self.extract(source)
        valid = self.validate(rows)
        return self.persist(valid)

    def extract(self, source):
        raise NotImplementedError

    def validate(self, rows):
        return rows

    def persist(self, rows):
        raise NotImplementedError
```

Prefer Strategy/composition when inheritance would couple implementations too tightly.

18. Iterator

Intent: Traverse a collection without exposing its representation.

Python iterators and generators provide language-native support.

```
def batches(rows, size):
    batch = []
    for row in rows:
        batch.append(row)
        if len(batch) == size:
            yield batch
            batch = []
    if batch:
        yield batch
```

19. State

Intent: Change behavior when an object's internal state changes.

Examples: Job lifecycle, payment state, order state, agent workflow.

```
CREATED -> PROCESSING -> SUCCEEDED
          \-> FAILED -> RETRYING
```

Every transition should define allowed source states, authorization, side effects, idempotency, and audit behavior.

20. Mediator

Intent: Centralize interactions among components so they do not depend directly on one another.

Example: An application mediator routes commands to handlers and publishes resulting domain events.

Trade-off: The mediator can become an opaque god object unless message contracts and handler ownership remain clear.

21. Memento

Intent: Capture and restore object state without exposing internals.

Examples: Undo, workflow checkpoints, draft recovery.

Trade-off: Snapshots can be expensive and may contain sensitive data. Define encryption, retention, schema compatibility, and replay semantics.

22. Visitor

Intent: Add operations across a stable object structure without modifying every element class.

Example: Generate audit, validation, and export representations across a stable policy AST.

Trade-off: Adding new element types requires updating visitors; therefore Visitor is strongest when element types are stable and operations change frequently.

23. Interpreter

Intent: Represent the grammar and evaluator for a small language.

Examples: Search filters, policy DSLs, rule expressions.

Security: Never pass an untrusted DSL directly to eval, SQL, or shell execution. Parse into an allow-listed AST, validate complexity, and enforce resource limits.

Modern Backend Patterns

Dependency Injection

Supply dependencies from the outside rather than constructing them in business logic.

```
class PaymentService:
    def __init__(self, repository, gateway):
        self.repository = repository
        self.gateway = gateway
```

Benefits include testability, explicit lifetimes, replaceable implementations, and centralized wiring.

Repository

Place persistence operations behind a domain-oriented interface.

```
class AccountRepository:
    async def get(self, account_id: str): ...
    async def save(self, account): ...
```

Do not hide every database capability behind generic CRUD if the application needs optimized queries, transactions, locking, or bulk operations.

Object Pool

Reuse expensive resources such as database connections.

Important controls:

- Maximum pool size and overflow
- Acquisition timeout
- Health validation
- Return/cleanup on every path
- Backpressure rather than unbounded connection creation

Lazy Initialization

Delay expensive creation until first use. It reduces startup work but moves failure and latency into a request path unless warmed and observed deliberately.

MVC / Layered Architecture

Separate transport/presentation, application/service logic, and persistence/domain concerns. FastAPI route handlers should normally coordinate validation and HTTP mapping rather than contain all business logic.

Event-Driven Architecture

Producers publish facts; consumers react asynchronously.

Production requirements include schema evolution, ordering scope, retries, dead-letter handling, idempotent consumers, observability, and transactional outbox when a database update and event must remain consistent.

CQRS

Separate command/write models from query/read models when read and write concerns genuinely differ.

CQRS does not require separate databases, event sourcing, or microservices. Start with logical separation and add infrastructure only when justified.

Event Sourcing

Store the sequence of domain events as the source of truth and rebuild state by replay.

Advantages:

- Complete change history
- Temporal reconstruction
- New projections from old events

Costs:

- Event schema evolution
- Replay and projection recovery
- Eventual consistency
- Privacy/deletion requirements
- Operational complexity

CQRS and event sourcing are independent patterns even though they are often combined.

High-Value Comparisons

Factory Method vs Abstract Factory vs Builder

Pattern	Select when
Factory Method	One product/interface varies
Abstract Factory	A compatible family of products varies together
Builder	One complex result is assembled through controlled steps

Adapter vs Facade vs Proxy vs Decorator

Pattern	Primary intent
Adapter	Change the interface
Facade	Simplify a subsystem
Proxy	Control access while preserving the interface
Decorator	Add composable behavior while preserving the interface

Strategy vs State

- **Strategy:** The caller/configuration selects an interchangeable algorithm.
- **State:** Internal lifecycle state drives behavior and allowed transitions.

The class diagrams may look similar; the intent and ownership of selection differ.

Observer vs Publish-Subscribe

- Observer is commonly an in-process subject-to-observer relationship.
- Publish-subscribe usually introduces a broker/topic and decouples producers from consumers across process boundaries.
- Distributed delivery requires durability, replay, ordering, deduplication, and failure semantics that the GoF Observer description does not provide.

Template Method vs Strategy

- Template Method varies steps through inheritance.
- Strategy varies behavior through composition.
- Prefer composition when algorithms must be changed independently at runtime or inheritance would create rigid coupling.

LLD Case Study: Extensible Transaction Processing

Requirements

- Create a transaction through an API.
- Validate business rules.
- Select a risk/pricing policy.
- Persist atomically and prevent duplicate requests.
- Publish an event after commit.
- Integrate with a replaceable external provider.
- Add tracing without embedding it in core logic.

Pattern Map

Force	Pattern/tool	Reason
Different risk algorithms	Strategy	Replace policy without changing the service
External provider has incompatible API	Adapter	Normalize vendor contract
Persistence boundary	Repository	Keep service independent of ORM details
Request as auditable work	Command	Stable operation ID and explicit input
Transaction lifecycle	State	Validate allowed transitions
Cross-cutting tracing	Decorator	Wrap behavior without changing domain logic
DB commit plus event publication	Transactional outbox	Avoid dual-write inconsistency
Dependency wiring/test replacement	Dependency injection	Explicit construction and lifetimes

```

FastAPI route
|
v
TransactionService
|-- Pricing/Risk Strategy
|-- Provider Adapter
|-- Repository
|-- Outbox Repository
|
v
single DB transaction
|
+--> transaction row
+--> outbox event
    |
    v
    publisher worker

```

Interview Trade-Offs

- A Strategy is justified when policy variants exist; otherwise a function is enough.
- Repository methods should reflect domain operations and important query semantics.
- The outbox solves database/event atomicity but adds a publisher, lag, retries, and monitoring.
- A Decorator is useful for tracing only if wrapper order and exception behavior stay understandable.
- Idempotency still needs a unique database constraint; an application lookup alone races.

Common Red Flags

- Pattern names without the changing force they solve.
- Singleton used as a substitute for dependency management.
- Factory for a class with one stable implementation.
- Repository that hides required transactions or creates N+1 queries.
- Observer used as if it guarantees distributed delivery.
- Event sourcing selected only because audit history is required.
- Strategy and State treated as interchangeable because their diagrams look similar.
- A pattern-heavy design with no simpler baseline.

60-Second Answer Template

The pattern's intent is __. The changing force is __, so I would separate __ from __. The main participants are __. In this system, an example is __. The benefit is __, while the cost is __. If that source of change does not exist yet, I would use the simpler design and introduce the pattern only when needed.

Quick Revision Checklist

Creation: Singleton, Factory Method, Abstract Factory, Builder, Prototype
Structure: Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy
Interaction: Observer, Strategy, Command, Chain, Template Method, Iterator,
State, Mediator, Memento, Visitor, Interpreter

Modern backend: DI, Repository, Object Pool, Lazy Initialization, MVC,
Event-Driven, CQRS, Event Sourcing, Transactional Outbox

Always state: intent -> participants -> example -> trade-off -> simpler alternative

Further Reading

- [Refactoring.Guru design patterns](#)
- [Python design-pattern examples](#)
- [Software Design Patterns Interview Guide PDF](#)